ALGORITMOS Y PROGRAMACIÓN II

Clase 4
Merge acumulador

Merge acumulador

Merge acumulador de listas en arreglos

Merge acumulador

Juan y Paula tienen listados los gastos de cada uno ordenados por rubro.

5/7 Impuestos 400

4/3 Impuestos 200

6/8 Peluquería 600

7/7 Ropa 1000

3/8 Peluquería 500

5/7 Ropa 1000

14/8 Ropa 500

Están interesados en conocer cuánto gastan entre los dos por cada rubro.

Merge acumulador

Juan y Paula tienen listados los gastos de cada uno ordenados por rubro.

5/7 Impuestos 400

4/3 Impuestos 200

6/8 Peluquería 600

7/7 Ropa 1000

Impuestos 600

Peluquería 1100

Ropa 2500

3/8 Peluquería 500

5/7 Ropa 1000

14/8 Ropa 500

Este es el resumen de gastos entre los dos por cada rubro.

Merge acumulador

Juan y Paula tienen listados los gastos de cada uno ordenados por rubro.

5/7 Impuestos 400

4/3 Impuestos 200

6/8 Peluquería 600

7/7 Ropa 1000

Impuestos 400

3/8 Peluquería 500

5/7 Ropa 1000

14/8 Ropa 500

Comparan el primer gasto que tienen en cada listado y se quedan con el “más chico”.

Merge acumulador

Juan y Paula tienen listados los gastos de cada uno ordenados por rubro.

4/3 Impuestos 200

6/8 Peluquería 600

7/7 Ropa 1000

Impuestos 600

3/8 Peluquería 500

5/7 Ropa 1000

14/8 Ropa 500

Comparan el primer gasto que tienen en cada listado y se quedan con el “más chico”. Coincide con el mismo del listado común, entonces lo suman.

Merge acumulador

Juan y Paula tienen listados los gastos de cada uno ordenados por rubro.

6/8 Peluquería 600

7/7 Ropa 1000

Impuestos 600

Peluquería 600

3/8 Peluquería 500

5/7 Ropa 1000

14/8 Ropa 500

Comparan el primer gasto que tienen en cada listado y se quedan con el “más chico”. No coincide con el del listado común, entonces lo incorporan.

Merge acumulador

Juan y Paula tienen listados los gastos de cada uno ordenados por rubro.

7/7 Ropa 1000

Impuestos 600

Peluquería 1100

3/8 Peluquería 500

5/7 Ropa 1000

14/8 Ropa 500

Comparan el primer gasto que tienen en cada listado y se quedan con el “más chico”. Coincide con el mismo del listado común, entonces lo suman.

Merge acumulador

Juan y Paula tienen listados los gastos de cada uno ordenados por rubro.

7/7 Ropa 1000

Impuestos 600

Peluquería 1100

Ropa 1000

5/7 Ropa 1000

14/8 Ropa 500

Comparan el primer gasto que tienen en cada listado y se quedan con el “más chico”. No coincide con el del listado común, entonces lo incorporan.

Merge acumulador

Juan y Paula tienen listados los gastos de cada uno ordenados por rubro.

Impuestos 600

Peluquería 1100

Ropa 2000

5/7 Ropa 1000

14/8 Ropa 500

Comparan el primer gasto que tienen en cada listado y se quedan con el “más chico”. Coincide con el mismo del listado común, entonces lo suman.

Merge acumulador

Juan y Paula tienen listados los gastos de cada uno ordenados por rubro.

Impuestos 600

Peluquería 1100

Ropa 2500

14/8 Ropa 500

Comparan el primer gasto que tienen en cada listado y se quedan con el “más chico”. Coincide con el mismo del listado común, entonces lo suman.

Merge acumulador

Juan y Paula tienen listados los gastos de cada uno ordenados por rubro.

Impuestos 600

Peluquería 1100

Ropa 2500

No tienen más gastos.

Merge acumulador

¿Qué pasa si comparten gastos N personas?

Merge acumulador

Supongamos que 5 personas comparten gastos y cada uno tiene su lista de gastos (tipo del gasto, fecha y monto) ordenados alfabéticamente por tipo de monto. Se necesita generar una única lista de impuestos con los costos totales.

```
Program acumulador;
uses GenericLinkedList;
Const cant = 5;
Type gasto = record
        tipo:string; fecha: string;monto: real;
             end;
     listaGastos = specialize LinkedList<gasto>;
   vectorListas = array [1..cant] of listaGastos;
     gastoComun = record
                  tipo:string; montoTotal: real;
             end;
listaGastosC = specialize LinkedList<gastoComun>;
Var v: vectorListas;
    listaUnica: listaGastosC;
Begin
 cargarGastos (v);
 mergeAcumulador (v, listaUnica);
End.
```

```
procedure cargarListaGastos (var l: listaGastos);

begin
  l:= listaGastos.create();
  ...{ código para cargar la lista }
end;

procedure cargarGastos (var v: vectorListas);
var i: integer;
begin
  for i:=1 to cant do
     cargarListaGastos (v[i]);
end;
```

Merge acumulador

```
procedure mergeAcumulador (v: vectorListas; var listaUnica: listaGastosC);
var gastoActual: gastoComun; min: gasto;
    i:integer;
begin
 listaUnica:= listaGastosC.create();
 for i:= 1 to cant do
    v[i].reset();
 minimo (v, min);
 while (min.tipo <> ‘zzz’) do
    begin
     gastoActual.tipo:= min.tipo;
     gastoActual.montoTotal:=0;
     while (min.tipo = gastoActual.tipo) do
     begin
        gastoActual.montoTotal:= gastoActual.montoTotal + min.monto;
        minimo (v, min);
     end;
     listaUnica.add(gastoActual);
    end;
end;
```

Merge acumulador

```
procedure minimo (var v: vectorListas; var min: gasto);
var indiceMin,i:integer;

begin
  min.tipo:= 'zzz';
  indiceMin:= -1;
  for i:= 1 to cant do
     if (not v[i].eol ()) then
       if (v[i].current().tipo <= min.tipo) then begin
                                  indiceMin:= i;
                                  min:= v[i].current();
                                  end;
  if (indiceMin <> -1) then v[indiceMin].next();
end;
```

Actividades en Máquina

ACTIVIDAD 1.

Implementar:

A. El módulo **cargarListaGastos** que lea los gastos aleatoriamente (de manera similar a como se generaban los inmuebles) y los almacene ordenados por tipo de gasto. La lectura finaliza cuando se lee monto 0.
B. Los módulos **cargarGastos()**, **mergeAcumulador()** y **minimo ()**. (sólo deberán transcribirlos de esta presentación).
C. Un módulo que reciba la “lista única” y retorne el tipo de gasto con mayor gasto.
D. Un programa que invoque a los módulos implementados y compruebe el correcto funcionamiento del mismo.

Actividades en Máquina

ACTIVIDAD 2

Un centro de distribución recibe paquetes de 6 lugares distintos. De cada paquete se conoce: código de paquete, código de lugar (entre 1 y 6), domicilio de envío y peso del paquete (Entre 1 Kg y 10 Kg en pasos de 500 gramos).

Implementar:

A. Un módulo llamado ***LeerPaquete()*** que lea los datos de paquetes aleatoriamente (de manera similar a como se generaban los inmuebles) y los almacene ordenados por peso y agrupados por código de lugar, en una estructura de datos adecuada. La lectura finaliza cuando se lee el código de paquete -1. ***Ya está resuelto!!!***

B. Un módulo que reciba la estructura generada en el punto a y retorne una estructura de datos donde se almacenen las duplas **(peso, cantidad)** de ese peso, ordenadas por peso.

C. Un programa que invoque a los módulos implementados y compruebe el correcto funcionamiento del mismo.

Actividades en Máquina

ACTIVIDAD 3

Un librería necesita procesar la información de las ventas de productos realizadas entre sus 4 sucursales.
Implementar:

A. Un módulo que lea las ventas y las retorne en una estructura de datos adecuada, las ventas ordenadas por código de producto y agrupados por sucursal. De cada venta se lee fecha de venta, código del producto vendido, código de sucursal y cantidad vendida. El ingreso de las ventas finaliza cuando se lee el código de sucursal 0.
B. Un módulo que reciba la estructura generada en el punto A. y retorne una estructura donde esté acumulada la cantidad total vendida para cada código de producto.
C. Un programa que invoque a los módulos implementados y compruebe el correcto funcionamiento del mismo.